

Criação do primeiro app Kotlin

Desenvolvimento de Software para Dispositivos Móveis

Prof. Me. Sergio Souza Novak — UniRV

10 de agosto de 2026



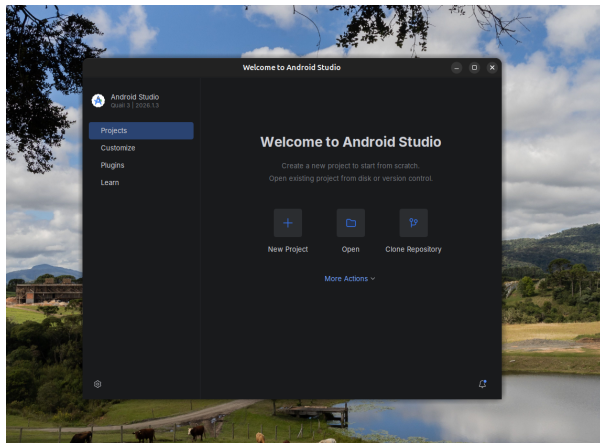
Sumário

- 1 Abrindo o Android Studio
- 2 Criando um novo projeto
- 3 Configuração do Projeto
- 4 Estrutura do Projeto

- 5 AndroidManifest.xml
- 6 Estrutura Detalhada do Projeto
- 7 Layout XML
- 8 Recursos do Projeto

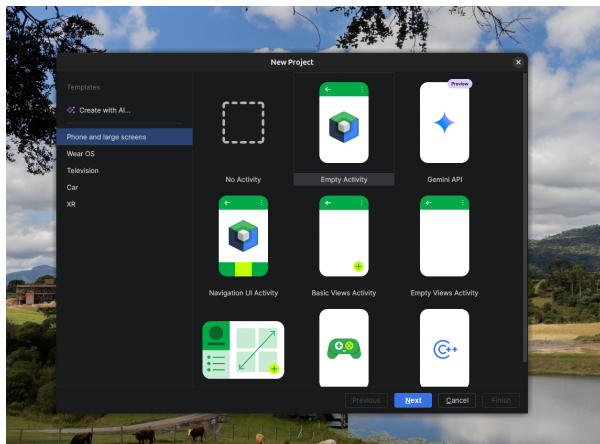
Abrindo o Android Studio

- Ao abrir o Android Studio, a tela de boas-vindas é exibida.
- Opções disponíveis:
 - **New Project** — criar um novo projeto.
 - **Open** — abrir um projeto existente.
 - **Clone Repository** — clonar um repositório Git.
- Clique em **New Project** para iniciar.



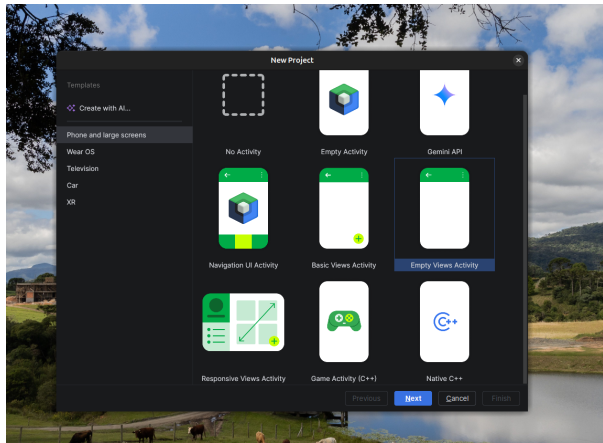
Seleção do Template — Empty Activity

- Na tela **New Project**, escolha a categoria **Phone and large screens**.
- Selecione o template **Empty Activity**.
- Este template cria um projeto com Jetpack Compose, a UI moderna do Android.
- Clique em **Next** para prosseguir.



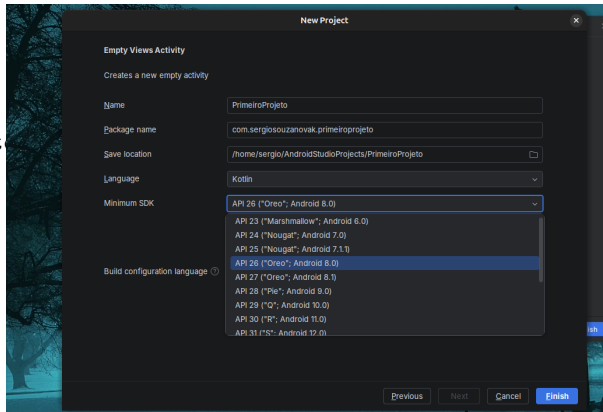
Seleção do Template — Empty Views Activity

- Alternativamente, selecione **Empty Views Activity**.
- Usa o sistema de Views tradicional (XML + Kotlin).
- Ideal para aprender os fundamentos do Android.
- Outras opções incluem:
 - Navigation UI Activity
 - Basic Views Activity
 - Gemini API (Preview)



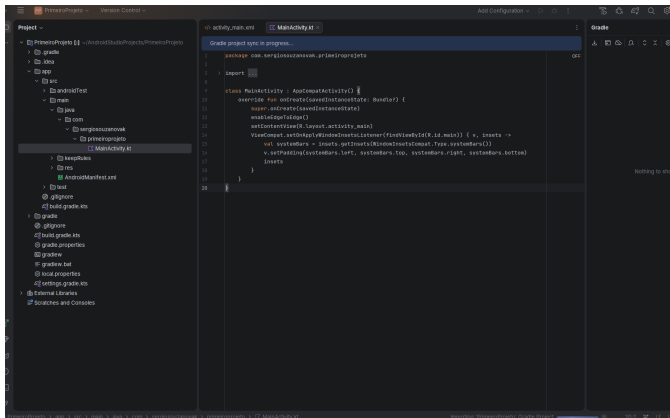
Configuração do Projeto

- **Name:** nome do app (ex: PrimeiroProjeto).
- **Package name:** identificador único (ex: com.sergiosouzanovak.primeiroprojeto).
- **Save location:** diretório onde o projeto será salvo.
- **Language:** selecione **Kotlin**.
- **Minimum SDK:** versão mínima do Android suportada (ex: API 26 — Android 8.0 Oreo).
- Clique em **Finish** para criar o projeto.



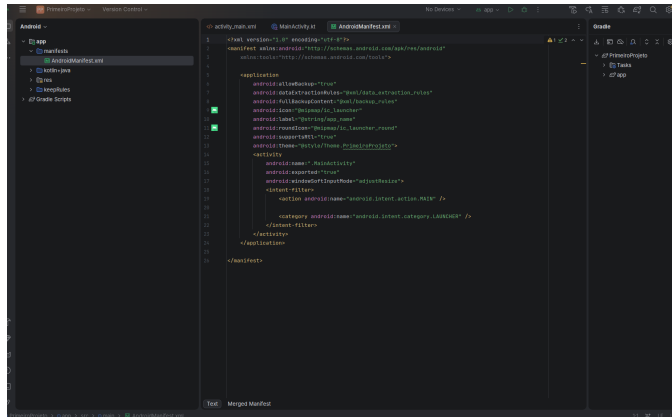
Estrutura do Projeto — MainActivity.kt

- O Android Studio abre o projeto e realiza o **Gradle Sync**.
- O arquivo **MainActivity.kt** é a Activity principal.
- Estrutura de pastas:
 - app/src/main/java/ — código Kotlin
 - app/src/main/res/ — recursos (layouts, strings, ícones)
 - build.gradle.kts — configuração de build
- A classe herda de AppCompatActivity.



AndroidManifest.xml

- O **AndroidManifest.xml** descreve o app para o sistema Android.
- Contém:
 - Nome do pacote
 - Permissões
 - Activities declaradas
 - Ícone e tema do app
 - Intent filters (qual Activity inicia o app)
- A tag `<intent-filter>` com MAIN e LAUNCHER define o ponto de entrada.



AndroidManifest.xml

O arquivo `AndroidManifest.xml` descreve informações essenciais sobre o aplicativo Android.

Estrutura básica

```
<manifest>
  <application>
    <activity>
      <intent-filter>
        ...
      </intent-filter>
    </activity>
  </application>
</manifest>
```

- **manifest**: arquivo de configuração do aplicativo.
- **application**: configurações gerais do aplicativo.
- **activity**: define uma tela/atividade do aplicativo.
- **intent-filter**: define como a Activity pode ser iniciada.

AndroidManifest: <application>

O elemento <application> contém configurações globais do aplicativo.

Exemplo

```
<application
    android:allowBackup="true"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportRtl="true"
    android:theme="@style/Theme.PrimeiroProjeto">
```

allowBackup Permite recursos de backup do aplicativo.

dataExtractionRules Define regras para extração/ transferência de dados em versões recentes do Android.

fullBackupContent Define regras de backup dos dados.

icon Define o ícone principal do aplicativo.

label Define o nome exibido para o aplicativo.

roundIcon Define uma versão circular do ícone.

AndroidManifest: Activity e Intent

A Activity representa uma unidade de interação do aplicativo, normalmente associada a uma tela.

Activity principal

```
<activity
    android:name=".MainActivity"
    android:exported="true"
    android:windowSoftInputMode="adjustResize">
```

name Indica a classe da Activity.

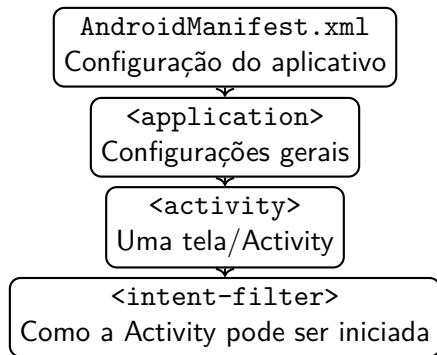
exported Indica se a Activity pode ser iniciada por componentes externos ao aplicativo.

windowSoftInputMode Define como a janela deve se comportar quando o teclado virtual aparecer.

Intent Filter

```
<intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
```

AndroidManifest: visão geral



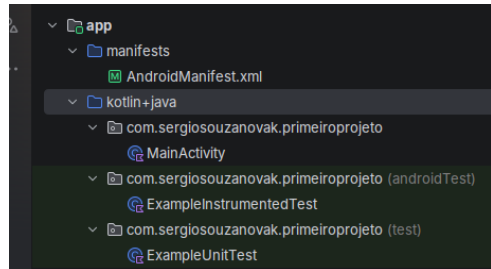
No exemplo

A sequência significa:

Aplicativo → **MainActivity** → **Ponto de entrada do aplicativo**

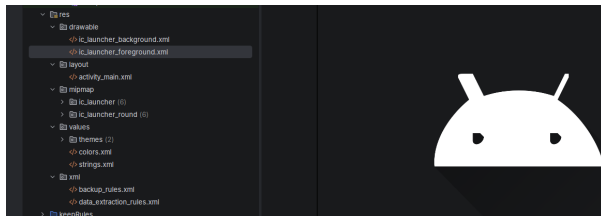
Pacotes do Projeto (kotlin+java)

- O código-fonte fica em kotlin+java.
- Três pacotes são criados automaticamente:
 - **Pacote principal** — contém a MainActivity e o código do app.
 - **androidTest** — testes de instrumentação (executados no dispositivo/emulador).
 - **test** — testes unitários (executados na JVM local).
- O nome do pacote segue a convenção:
`com.empresa.app`.



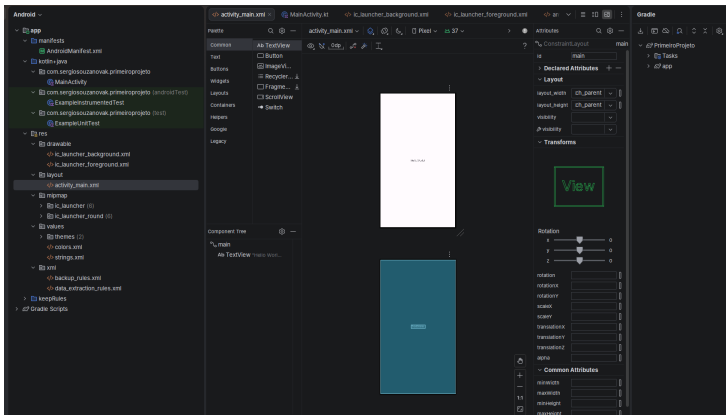
Pasta de Recursos (res/)

- A pasta `res/` contém todos os recursos do app:
 - **drawable** — imagens e gráficos vetoriais (ícones).
 - **layout** — arquivos XML de layout (`activity_main.xml`).
 - **mipmap** — ícones do app em várias resoluções.
 - **values** — constantes: cores, strings, temas.
 - **xml** — configurações de backup e extração de dados.



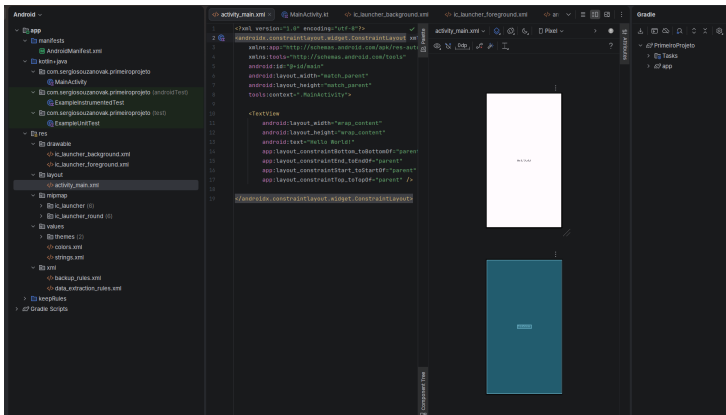
Layout Editor — Modo Visual

- O **Layout Editor** permite criar interfaces visualmente.
- Componentes disponíveis na **Palette**: Button, TextView, ImageView, etc.
- O **Component Tree** mostra a hierarquia de views.
- O painel **Attributes** permite configurar propriedades.
- Preview em tempo real do layout.



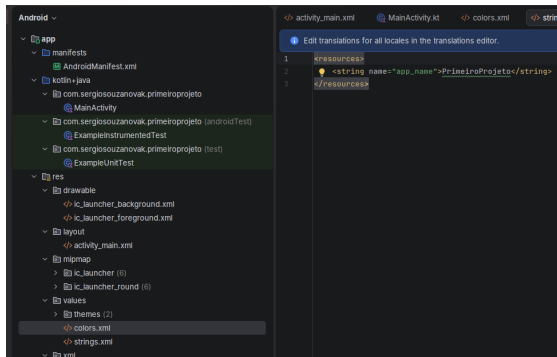
Layout XML — Modo Código

- O arquivo `activity_main.xml` define a interface.
- Usa **ConstraintLayout** como layout raiz.
- Contém um **TextView** com o texto “Hello World!”.
- Constraints posicionam o `TextView` centralizado na tela.
- O preview é exibido ao lado do código XML.



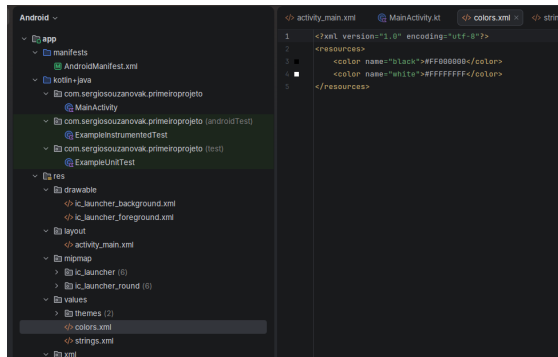
Recursos: strings.xml e colors.xml

strings.xml



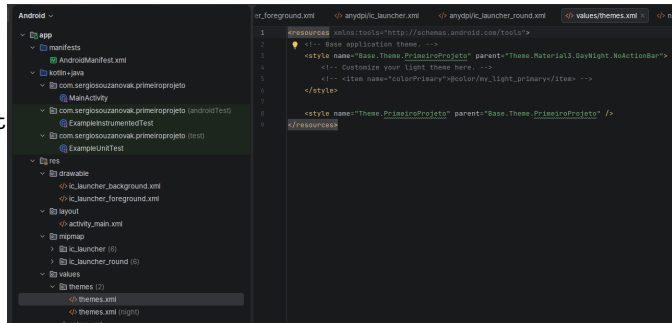
- Define textos reutilizáveis do app.
- `app_name`: nome exibido do aplicativo.
- Facilita a internacionalização (i18n).

colors.xml



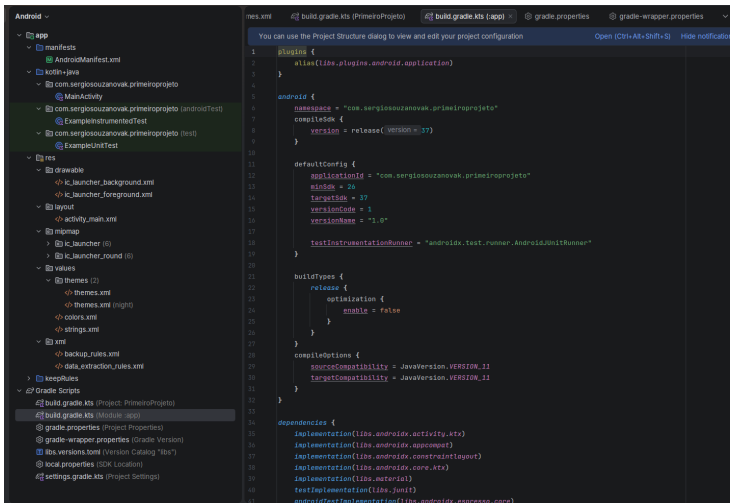
- Define as cores do app em formato hexadecimal.
- Cores padrão: `black` e `white`.
- Referenciadas como `@color/nome`.

- O arquivo `themes.xml` define o tema visual do app.
- Herda de **Material3**
(`Theme.Material3.DayNight.NoActionBar`)
- Permite personalizar cores primárias, secundárias, etc.
- Versão **night** para modo escuro (tema separado).
- O tema é referenciado no `AndroidManifest.xml`.



build.gradle.kts — Configuração do Módulo

- O **build.gradle.kts (:app)** configura o módulo.
- Seções principais:
 - **plugins** — plugin Android
 - **android** — SDK, namespace, versões
 - **defaultConfig** — applicationId, minSdk, targetSdk
 - **buildTypes** — configurações de release
 - **compileOptions** — versão do Java

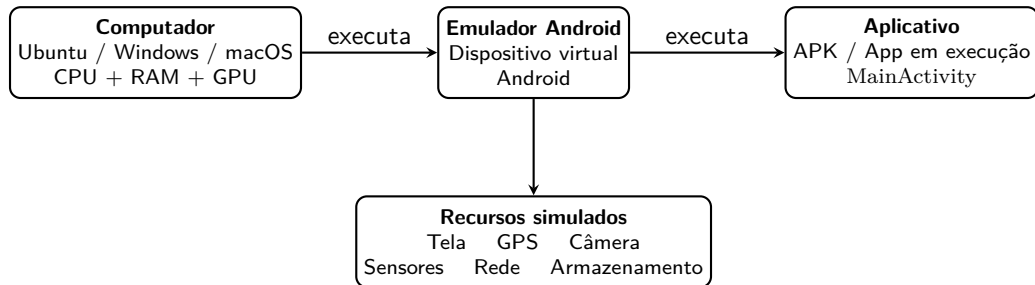


build.gradle.kts — Dependências

- O bloco **dependencies** lista as bibliotecas utilizadas.
- Dependências padrão incluem:
 - **activity.ktx** — extensões para Activity
 - **appcompat** — compatibilidade
 - **constraintlayout** — layout flexível
 - **core.ktx** — extensões Kotlin
 - **material** — Material Design
- Testes: junit, espresso.
- Gerenciadas via **Version Catalog** (**libs.versions.toml**).



Emulador Android

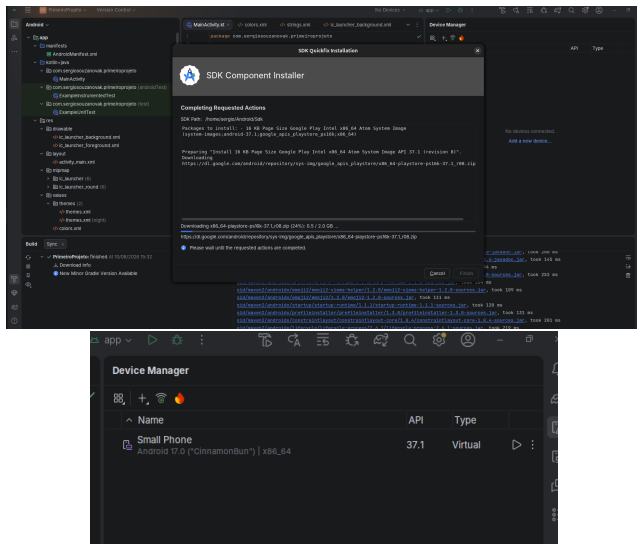


Definição

O **emulador Android** é um software que simula um dispositivo Android dentro do computador, permitindo desenvolver e testar aplicativos sem utilizar um dispositivo físico.

Configurando o Emulador — SDK e Device Manager

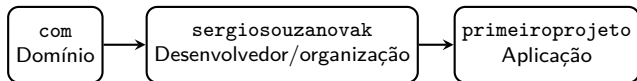
- Antes de executar, é necessário instalar a **System Image** do Android.
- O Android Studio baixa automaticamente via **SDK Component Installer**.
- Após a instalação, o **Device Manager** exibe os dispositivos virtuais.
- No exemplo: **Small Phone** com API 37.1 (Android 17.0).



Código Android: package

Primeira linha

```
package com.sergiosouzanovak.primeiroprojeto
```



- Define o **pacote** ao qual a classe pertence.
- Organiza as classes e recursos do projeto.
- Ajuda a evitar conflitos de nomes entre aplicativos.
- É utilizado pelo Android para identificar o código da aplicação.

Exemplo

O pacote funciona como o “endereço” da classe dentro do projeto.

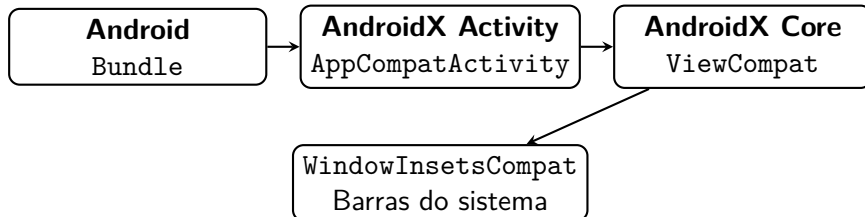
Código Android: import

O comando `import` permite utilizar classes e funções disponíveis em outras bibliotecas.

Imports do projeto

```
import android.os.Bundle

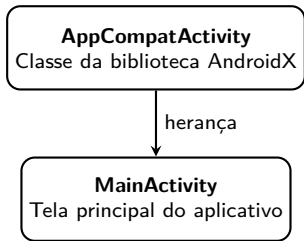
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
```



Código Android: MainActivity

Declaração da classe

```
class MainActivity : AppCompatActivity() {  
    ...  
}
```

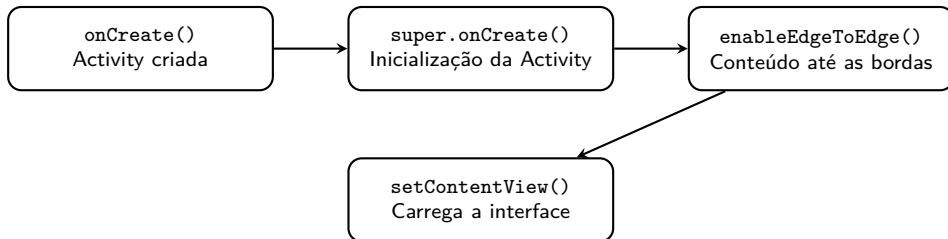


- `class`: declara uma classe Kotlin.
- `MainActivity`: nome da classe.
- `: AppCompatActivity()`: indica **herança**.
- A `Activity` representa uma tela/interação do aplicativo.

Código Android: onCreate()

Método de inicialização

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    enableEdgeToEdge()  
    setContentView(R.layout.activity_main)  
}
```



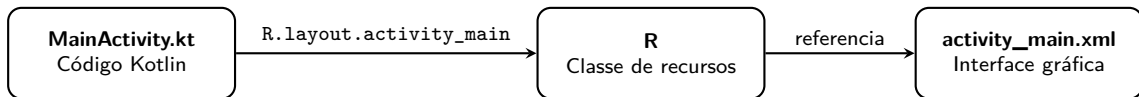
- `override`: sobrescreve um método da classe pai.

- `onCreate()`: executado quando a Activity é criada.

setContentView() e o layout XML

Código Kotlin

```
setContentView(R.layout.activity_main)
```



- R é uma classe gerada automaticamente pelo Android.
- layout representa a pasta de layouts.
- activity_main identifica o arquivo XML.
- O XML define a interface visual.
- O Kotlin controla o comportamento da interface.

Resumo

Kotlin = comportamento

XML = interface

Código

```
enableEdgeToEdge()

ViewCompat.setOnApplyWindowInsetsListener(
    findViewById(R.id.main)
) { v, insets ->

    val systemBars =
        insets.getInsets(
            WindowInsetsCompat.Type.systemBars()
        )

    v.setPadding(
        systemBars.left,
        systemBars.top,
        systemBars.right,
        systemBars.bottom
    )

    insets
}
```

MainActivity: fluxo de execução



Em resumo

MainActivity.kt é responsável por inicializar a Activity, carregar a interface e configurar o comportamento da tela.

Conceitos iniciais

- **Define a estrutura da interface gráfica**
 - Como os elementos se comportam e se relacionam
- **Todos os elementos dentro do layout respeitam a hierarquia de View e ViewGroup**

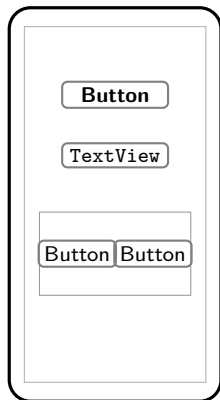
View

Geralmente representam elementos da interface com os quais o usuário pode interagir
Button, EditText, Switch,
ImageView, CheckBox

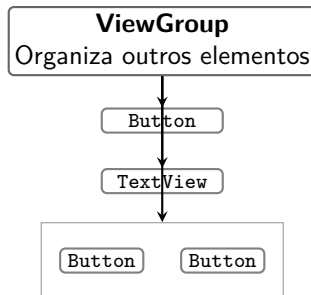
ViewGroup

Elementos de layout que organizam e definem o comportamento de outros elementos
`LinearLayout`, `RelativeLayout`,
`ConstraintLayout`, `RecyclerView`

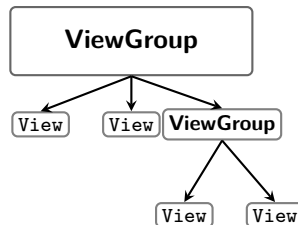
Conceitos iniciais



Interface gráfica



View e ViewGroup



Hierarquia de Views

Conceito fundamental

View representa um elemento da interface.

ViewGroup organiza e contém outras Views.

Boas práticas

- A quantidade de elementos influencia o **processamento** e o **tempo de carregamento** da tela.
- Quanto maior a quantidade de elementos e níveis de hierarquia, maior tende a ser o custo de processamento.

```
<LinearLayout>
  <Button />
  <TextView />
  <LinearLayout>
    <Button />
    <Button />
  </LinearLayout>
</LinearLayout>
```

Preferível

```
<LinearLayout>
  <LinearLayout>
    <Button />
  </LinearLayout>
  <LinearLayout>
    <TextView />
  </LinearLayout>
  <LinearLayout>
    <LinearLayout>
      <Button />
    </LinearLayout>
  </LinearLayout>
</LinearLayout>
```

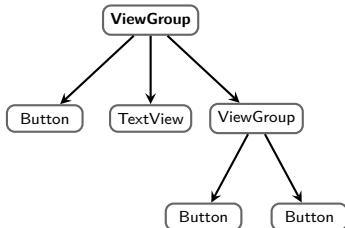
Evitar excesso de hierarquia

Boas práticas: hierarquia de Views

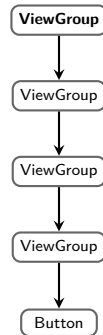
Regra geral

Evite hierarquias de Views desnecessariamente profundas.

- Menos níveis de hierarquia podem simplificar o processamento.
- Evite ViewGroups utilizados apenas para agrupar elementos.
- Prefira estruturas de layout mais simples quando possível.



Hierarquia simples



Código Kotlin: Hierarquia de Views

- Embora a hierarquia seja desenhada no XML, podemos manipulá-la via Kotlin.
- Adicionar Views dinamicamente a um ViewGroup custa processamento.

```
val layoutPai = findViewById<LinearLayout>(R.id.layoutPrincipal)

// Criando um botao dinamicamente
val novoBotao = Button(this).apply {
    text = "Botao Dinamico"
}

// Adicionando o botao a hierarquia existente
layoutPai.addView(novoBotao)

// Removendo uma View da hierarquia
layoutPai.removeView(novoBotao)
```

Density-independent pixels

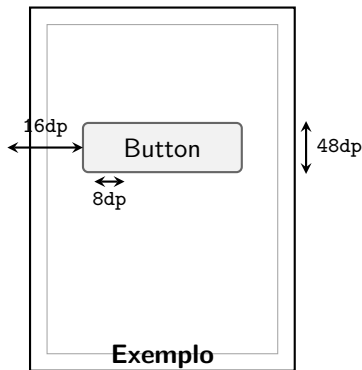
dp

Density-independent pixels

Unidade utilizada para definir as dimensões dos elementos da interface.

Pode ser utilizada para:

- Largura e altura dos componentes
- Margens e espaçamentos
- Padding
- Tamanho de áreas da interface



Botão com altura de 48dp,
margem de 16dp
e padding de 8dp.

Código Kotlin: Trabalhando com sp

- Ao definir o tamanho do texto via código Kotlin, o padrão já é sp.

```
val textView = findViewById<TextView>(R.id.meuTexto)

// Define o tamanho da fonte. O Android já interpreta o '18f' como SP
// por padrão ao atribuir a propriedade textSize
textView.textSize = 18f

// Metodo alternativo, para ser explicito e garantir que e SP:
textView.setTextSize(TypedValue.COMPLEX_UNIT_SP, 18f)

// Lendo tamanho do arquivo dimens.xml e aplicando
val tamanhoTexto = resources.getDimension(R.dimen.tamanho_fonte)
textView.setTextSize(TypedValue.COMPLEX_UNIT_PX, tamanhoTexto)
```

Scale-independent pixels

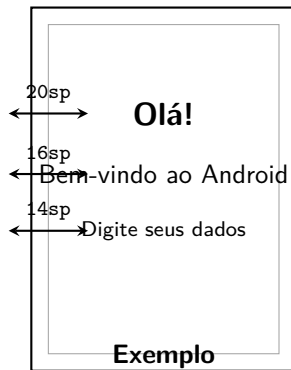
sp

Scale-independent pixels

Unidade recomendada para definir o tamanho dos textos da interface.

Principais características:

- Utilizado para tamanho de fontes
- Considera a escala de fonte do sistema
- Permite ampliar o texto conforme a preferência do usuário
- Favorece acessibilidade e legibilidade



Título: 20sp

Texto normal: 16sp

Texto auxiliar: 14sp

Código Kotlin: Trabalhando com dp

- No Kotlin, muitas funções exigem dimensões em **pixels puros (px)**, não em dp.
- É preciso converter dp para px considerando a densidade da tela.

```
// Valor desejado em dp
val valorEmDp = 16f

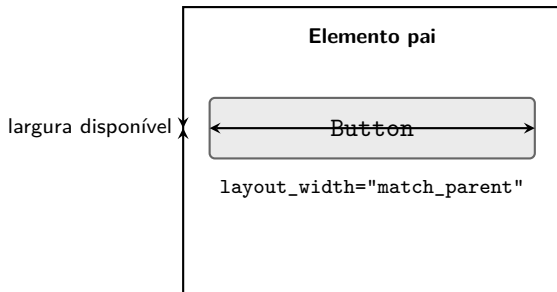
// Convertendo dp para pixels (px) de acordo com a tela do usuario
val valorEmPx = TypedValue.applyDimension(
    TypedValue.COMPLEX_UNIT_DIP, // Indicando que a origem e DP
    valorEmDp,
    resources.displayMetrics
).toInt()

// Aplicando padding (espera pixels como parametro)
val textView = findViewById<TextView>(R.id.meuTexto)
textView.setPadding(valorEmPx, valorEmPx, valorEmPx, valorEmPx)
```

Boas práticas: `match_parent` e `wrap_content`

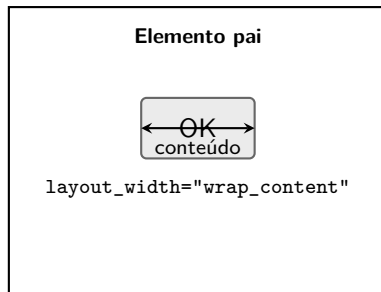
`match_parent`

Ocupa todo o espaço disponível do elemento pai.



`wrap_content`

Ocupa apenas o espaço necessário para acomodar o conteúdo.



Regra prática

`match_parent` = preencher o espaço disponível

`wrap_content` = ajustar ao conteúdo

Código Kotlin: match_parent e wrap_content

- No Kotlin, manipulamos as dimensões através da classe LayoutParams.

```
val botao = findViewById<Button>(R.id.meuBotao)

// Criando parametros de layout (Largura e Altura)
val parametros = LinearLayout.LayoutParams(
    LinearLayout.LayoutParams.MATCH_PARENT, // Largura total
    LinearLayout.LayoutParams.WRAP_CONTENT // Altura do conteudo
)

// Opcional: Aplicando margens aos parametros
parametros.setMargins(16, 16, 16, 16)

// Atribuindo os parametros ao botao
botao.layoutParams = parametros
```

Boas práticas no XML

- Elementos escritos com “self-closing tags”

```
<Button />
```



```
<Button></Button>
```



- Elementos são “case sensitive”

```
<Button />
```



```
<button />
```



Boas práticas

Tags devem respeitar a sintaxe e a capitalização esperadas pelo Android.